

Release 1 Story Estimates

“The project follows an Agile development approach and focuses on implementing a digital marketplace where users can post service requests and receive competitive bids from local providers.”

“The 1st release, totaling 46 story points, covers core functionality required for the system to operate, including login and registration, dashboards, posting service requests, viewing and comparing bids, confirming bookings, provider profile setup, booking confirmation with calendar blocking, and basic admin and moderation tasks.”

Source: COMP313 - Team 6 - Methodologies.pdf

Story / Screen ID	Story Summary	Estimated Points
SS-R1-1	Login / Register	5
SS-R1-2	Create service request	5
SS-R1-3	View bids & compare	3
SS-R1-4	Confirm booking	8
SS-R1-1	Provider profile setup	5
SS-R1-2	View open requests	3
SS-R1-3	Booking confirmation + calendar block	8
AD-R1-1	Admin basic dashboard	3
AD-R1-2	User management	3
AD-R1-3	Ratings/Review Moderation	3

Release 1 Total: 46 points

Disaggregation tables for Release 1 stories

“The project follows an Agile development approach and focuses on implementing a digital marketplace where users can post service requests and receive competitive bids from local providers.”

“Core system components include service request management, provider bidding, automated calendar updates to prevent double bookings, and a rating and review system to promote service quality.”

Source: COMP313 - Team 6 - Methodologies.pdf

Below are the disaggregation tables for **each Release 1 story**. Each table follows the requested format (**Task | Who | Estimate | Actual**). The **Estimate** column for the subtasks of a story sums to the story point value shown in the original Release 1 table. The **Who** column uses **Dev1 ... Dev7**. The **Actual** column is left blank for later fill-in. Tasks are ordered in the recommended implementation sequence and include RBAC/auth middleware tasks where relevant. Some subtasks are grouped so the same developer performs related work.

SS-R1-1 — Login / Register (5)

Task	Who	Estimate	Actual
Design auth flow & UI wireframes	Dev1	1	
Implement backend auth endpoints (register/login)	Dev2	1	
Implement RBAC middleware (roles: Customer, Provider, Admin)	Dev2	1	
Frontend login/register forms & validation	Dev3	1	
Integration testing & bugfixes (auth + RBAC)	Dev1	1	

SS-R1-2 — Create service request (5)

Task	Who	Estimate	Actual
UX: service request form design & fields	Dev3	1	
Backend: ServiceRequest model + API create endpoint	Dev4	1	
Frontend: form implementation & client validation	Dev3	1	
Server-side validation & permissions check (RBAC)	Dev2	1	
End-to-end test & small polish (notifications stub)	Dev5	1	

SS-R1-3 — View bids & compare (3)

Task	Who	Estimate	Actual
Backend: bids list API (filter by request)	Dev4	1	
Frontend: bids list UI + compare view	Dev3	1	
Sorting/filtering & basic accessibility fixes	Dev5	1	

SS-R1-4 — Confirm booking (8)

Task	Who	Estimate	Actual
Backend: booking model + confirm endpoint	Dev4	2	
Calendar block logic (atomic booking + conflict check)	Dev2	2	
Frontend: confirm booking flow & confirmation UI	Dev3	1	
Notifications (confirmation email stub / UI notice)	Dev6	1	
Integration tests (race conditions, concurrency)	Dev2	1	
End-to-end acceptance test & bugfixes	Dev1	1	

Provider profile setup — (SS-R1-1 duplicate ID) (5)

Task	Who	Estimate	Actual
Profile data model + API endpoints	Dev4	1	
Frontend: provider profile setup UI	Dev3	1	
Upload/attachments handling (images, docs)	Dev6	1	
RBAC: provider-only profile actions enforcement	Dev2	1	
Tests & UX polish (profile preview)	Dev5	1	

View open requests — (SS-R1-2 duplicate ID) (3)

Task	Who	Estimate	Actual
Backend: open requests query API (filters)	Dev4	1	
Frontend: provider dashboard list of open requests	Dev3	1	
Performance tuning & pagination	Dev6	1	

Booking confirmation + calendar block — (SS-R1-3 duplicate ID) (8)

Task	Who	Estimate	Actual
Robust calendar model + availability store	Dev4	2	
Atomic booking transaction & conflict detection	Dev2	2	
Frontend calendar UI (provider view)	Dev3	1	
Provider calendar block automation (backend job)	Dev2	1	
Edge-case tests (overlap, daylight savings)	Dev1	1	
Logging/observability for booking flows	Dev7	1	

AD-R1-1 — Admin basic dashboard (3)

Task	Who	Estimate	Actual
Backend: admin summary endpoints (counts)	Dev4	1	
Frontend: admin dashboard UI (summary cards)	Dev5	1	
Access control & admin-only route enforcement	Dev2	1	

AD-R1-2 — User management (3)

Task	Who	Estimate	Actual
Backend: user list, suspend/reactivate endpoints	Dev4	1	
Frontend: user management UI (search, actions)	Dev5	1	
Audit logging & permission checks	Dev7	1	

AD-R1-3 — Ratings/Review Moderation (3)

Task	Who	Estimate	Actual
Backend: ratings model + moderation flags	Dev4	1	
Frontend: moderation queue UI for admins	Dev5	1	
Moderation actions + tests (hide, reinstate)	Dev7	1	

Notes on assignment strategy

- **Seven developers** (Dev1–Dev7) are used; assignments reflect domain fit and continuity (e.g., Dev2 focuses on RBAC and concurrency-critical backend work; Dev3 focuses on frontend UI tasks; Dev4 handles core backend models/APIs; Dev5 handles admin and UX polish; Dev6 handles uploads and performance; Dev7 handles logging/audit and some admin tasks).
 - **Estimates** are integers and the **sum of subtask estimates equals the story point** for each story.
 - **Order**: tables are sorted in a logical implementation order (auth & registration first, then request creation, bids, booking/ calendar, then admin features).
 - **RBAC/auth middleware** is included in Login/Register and applied to relevant stories (service request creation, provider profile, admin routes, moderation).
 - No developer is overloaded unrealistically; however, assignments are intentionally **uneven** to reflect skill specialization and the instruction that distribution should not aim to be perfectly even.
-

Acceptance tests for Release 1 — each story ID with tests for its disaggregated subtasks

Story / Screen ID	Acceptance test(s) (all subtasks, semicolon-separated)	Contributing Developer(s)
SS-R1-1 — Login / Register	Wireframes match spec; register creates user, rejects duplicate email, stores hashed password; login succeeds with correct credentials and fails with wrong ones; RBAC roles assigned and protected routes enforce roles; logout clears session; end-to-end auth flow passes.	Dev1; Dev2; Dev3
SS-R1-2 — Create service request	Form fields present and validated; POST /service-requests creates record with required fields; server rejects invalid/missing fields; unauthenticated/non-customer cannot create; created request appears in customer dashboard and DB.	Dev3; Dev4; Dev2; Dev5
SS-R1-3 — View bids & compare	Providers can submit bids; GET /requests/{id}/bids returns all bids; customer sees bids list; sort by price/rating works; compare view shows bid details and provider profile; UI responsive and accessible.	Dev4; Dev3; Dev5
SS-R1-4 — Confirm booking	Selecting a bid creates Booking record with correct scheduled time; provider calendar is blocked for that slot; confirmation notification generated (UI/email stub); booking status transitions to Confirmed; booking appears in both dashboards.	Dev4; Dev2; Dev3; Dev6
SS-R1-1 — Provider profile setup	Provider can save/edit profile; GET returns saved data; uploads (image/docs) stored and retrievable; profile visible in provider listings and bid views; provider-only actions enforced by RBAC.	Dev4; Dev3; Dev6; Dev5
SS-R1-2 — View open requests	GET /open-requests returns filtered results by service type/location/date; provider dashboard lists relevant open requests; filters and pagination work; providers cannot access requests outside permissions.	Dev4; Dev3; Dev6
SS-R1-3 — Booking confirmation +	Calendar shows availability and blocked slots; confirm booking performs atomic DB transaction; overlapping	Dev2; Dev4;

Story / Screen ID	Acceptance test(s) (all subtasks, semicolon-separated)	Contributing Developer(s)
calendar block	confirmations rejected; blocked slot visible in provider calendar; DST/timezone edge cases handled; booking flow logged.	Dev1; Dev7
AD-R1-1 — Admin basic dashboard	Admin summary endpoints return correct counts; dashboard UI displays matching metrics; admin-only routes blocked for non-admins; dashboard updates after create/suspend actions.	Dev4; Dev5; Dev2
AD-R1-2 — User management	Admin can search/filter users; suspend/reinstate actions work and create audit log entries; suspended user cannot log in or create requests; UI shows confirmations and error states.	Dev4; Dev5; Dev7
AD-R1-3 — Ratings/Review Moderation	Customer can submit rating linked to booking; new reviews appear in moderation queue; admin can hide/reinstate reviews; hidden reviews not visible to public; only admins can moderate.	Dev4; Dev5; Dev7

SS-R1-1 — Login / Register (5)

1. **Test with inputs:** Register form submitted with valid email, strong password, role = Customer.
Expected outcome: New user record created; response 201; password stored hashed; role set to Customer.
 2. **Test with inputs:** Register form submitted with an email already in use.
Expected outcome: Response 409 (conflict) with message about duplicate email; no new user created.
 3. **Test with inputs:** Login request with correct email and password.
Expected outcome: Response 200 with auth token/session cookie; subsequent protected route accessible.
 4. **Test with inputs:** Login request with correct email and wrong password.
Expected outcome: Response 401 (unauthorized); no token issued.
 5. **Test with inputs:** Access admin-only endpoint using Customer token.
Expected outcome: Response 403 (forbidden); RBAC middleware blocks access.
 6. **Test with inputs:** Logout request using valid session/token.
Expected outcome: Session invalidated; protected route returns 401 after logout.
-

SS-R1-2 — Create service request (5)

1. **Test with inputs:** Customer submits service request with all required fields (serviceType, location, preferredTime, description).
Expected outcome: Response 201; ServiceRequest record created with correct fields; appears in customer dashboard.
2. **Test with inputs:** Customer submits service request missing required field (serviceType).
Expected outcome: Response 400 (bad request) with validation error; no record created.
3. **Test with inputs:** Unauthenticated user attempts to POST /service-requests.
Expected outcome: Response 401; request rejected.
4. **Test with inputs:** Authenticated Provider token attempts to create a service request.
Expected outcome: Response 403; only Customers allowed to create requests.
5. **Test with inputs:** Create request then GET customer dashboard.
Expected outcome: Newly created request visible in dashboard list and matches DB entry.

SS-R1-3 — View bids & compare (3)

1. **Test with inputs:** Provider submits a bid for an existing request (price, availableTime).
Expected outcome: Response 201; Bid record linked to request and provider; timestamp recorded.
2. **Test with inputs:** Customer requests GET /requests/{id}/bids.
Expected outcome: Response 200 with list of bids for that request; includes price, provider profile, timestamp.
3. **Test with inputs:** Customer sorts bids by price ascending and by provider rating.
Expected outcome: Bids returned in correct order for each sort; compare view shows full bid details and provider profile.
4. **Test with inputs:** Provider attempts to view bids for a request they are not permitted to see.
Expected outcome: Response 403 or empty list per permissions; no unauthorized data exposure.

SS-R1-4 — Confirm booking (8)

1. **Test with inputs:** Customer selects a bid and sends confirm request (bidID).
Expected outcome: Response 201; Booking record created with scheduledTime from bid; status = Confirmed.
2. **Test with inputs:** Confirm booking triggers calendar block for provider at scheduledTime.
Expected outcome: Provider calendar shows blocked slot; slot unavailable for other bookings.
3. **Test with inputs:** Simulate two concurrent confirm requests for the same bid/slot.
Expected outcome: Only one confirm succeeds; other request returns 409 or 409-like conflict; no double booking.
4. **Test with inputs:** Confirm booking with invalid bidID or mismatched request.
Expected outcome: Response 400/404; no booking created.
5. **Test with inputs:** After confirm, check customer and provider dashboards.
Expected outcome: Booking appears in both dashboards with status Confirmed and correct details.

6. **Test with inputs:** Confirm booking triggers notification stub (UI/email).
Expected outcome: Notification record created or UI shows confirmation message; email stub logged.
-

SS-R1-1 (Provider profile setup) — Provider profile setup (5)

1. **Test with inputs:** Provider submits profile data (name, serviceType, bio) and saves.
Expected outcome: Response 200/201; ProviderProfile stored and retrievable via GET.
 2. **Test with inputs:** Provider uploads profile image/document.
Expected outcome: Upload accepted; file stored; GET profile returns link/reference to stored file.
 3. **Test with inputs:** Customer views provider profile from bid or provider listing.
Expected outcome: Profile data and image visible to customer; no sensitive fields exposed.
 4. **Test with inputs:** Non-provider user attempts provider-only profile actions.
Expected outcome: Response 403; RBAC prevents action.
 5. **Test with inputs:** Provider edits profile and saves changes.
Expected outcome: Changes persisted; GET returns updated profile.
-

SS-R1-2 (View open requests) — View open requests (3)

1. **Test with inputs:** Provider requests GET /open-requests with no filters.
Expected outcome: Response 200 with list of open requests relevant to provider (not including closed/assigned).
 2. **Test with inputs:** Provider applies filters (serviceType, location, date).
Expected outcome: Returned list matches filters; results count correct.
 3. **Test with inputs:** Provider requests paginated page 2 with pageSize parameter.
Expected outcome: Response 200 with correct subset of requests and pagination metadata.
 4. **Test with inputs:** Provider attempts to access requests outside their permission scope.
Expected outcome: Those requests are not returned; access controlled.
-

SS-R1-3 (Booking confirmation + calendar block) — Booking confirmation + calendar block (8)

1. **Test with inputs:** Create booking and verify calendar model stores blocked slot.
Expected outcome: Calendar availability shows blocked slot for provider at scheduledTime.
 2. **Test with inputs:** Attempt to create overlapping booking for same provider and time.
Expected outcome: Response 409/conflict; booking rejected.
 3. **Test with inputs:** Confirm booking across DST boundary or different timezones.
Expected outcome: ScheduledTime normalized correctly; no overlap due to timezone miscalculation.
 4. **Test with inputs:** Run atomic transaction test (simulate DB failure mid-flow).
Expected outcome: Either booking and calendar block both succeed or neither; no partial state.
 5. **Test with inputs:** Provider calendar UI displays blocked slot after booking.
Expected outcome: Blocked slot visible and not selectable for new bookings.
 6. **Test with inputs:** Check logs/observability entries for booking flow.
Expected outcome: Audit/log entry created with booking ID, user IDs, timestamp, and outcome.
-

AD-R1-1 — Admin basic dashboard (3)

1. **Test with inputs:** Admin requests dashboard summary endpoint.
Expected outcome: Response 200 with counts for users, open requests, bookings; values match DB.
 2. **Test with inputs:** Non-admin attempts to access admin dashboard route.
Expected outcome: Response 403; access denied.
 3. **Test with inputs:** Create a new user and refresh dashboard.
Expected outcome: User count increments and dashboard reflects new total.
-

AD-R1-2 — User management (3)

1. **Test with inputs:** Admin searches users by email and name.
Expected outcome: Search returns matching users with correct metadata.

2. **Test with inputs:** Admin suspends a user account.
Expected outcome: Response 200; user status = suspended; audit log entry created.
 3. **Test with inputs:** Suspended user attempts to log in or create a request.
Expected outcome: Login blocked or actions rejected; appropriate error returned.
 4. **Test with inputs:** Admin reinstates suspended user.
Expected outcome: User status = active; audit log entry created; user can log in again.
-

AD-R1-3 — Ratings/Review Moderation (3)

1. **Test with inputs:** Customer submits rating and review after a completed booking.
Expected outcome: Response 201; Rating linked to booking and customer; appears in DB.
 2. **Test with inputs:** Admin views moderation queue.
Expected outcome: New reviews appear in queue with metadata (bookingID, customerID, score, comment).
 3. **Test with inputs:** Admin hides a review from public view.
Expected outcome: Review state = hidden; public endpoints do not return hidden review.
 4. **Test with inputs:** Admin reinstates a hidden review.
Expected outcome: Review state = visible; public endpoints return the review again.
 5. **Test with inputs:** Non-admin attempts moderation action.
Expected outcome: Response 403; action blocked.
-